1.- **b.** `int[] arr = {1, 2, 3};` **(Válida)**: Es la forma abreviada (initializer shorthand) de inicializar un arreglo cuando la declaración y la asignación se realizan en la misma línea.
c. `int[] arr = new int[] {1, 2, 3};` **(Válida)**: Utiliza una expresión de creación de arreglo anónimo con sus valores iniciales explícitos. Es útil cuando se quiere inicializar el arreglo con valores conocidos pero en una línea separada de la declaración (o como argumento de un método).
d. `int[] arr = new int[3];` **(Válida)**: Crea un arreglo especificando su tamaño (`3`), inicializando automáticamente todos sus elementos con su valor por defecto (que es `0` para los tipos `int`).
2.- D. result comienza en 0.En cada iteración se suman los valores de la diagonal: 0 + 1 + 5 + 9 = 15.
3.- C. Cuando `i = 0, j = 0`: Accede a `matrix[0][0]`, que es **`1`**.

- Cuando `i = 1, j = 1`: Accede a `matrix[1][1]`, que es **`5`**.
- Cuando `i = 2, j = 2`: Accede a `matrix[2][2]`, que es **`9`**.

Salida: Se imprime cada valor seguido de un espacio (`System.out.print(matrix[i][j] + " ")`), dando como resultado **`1 5 9`**
4.- C. Accede a la fila 1 ({0, 1}), la cual tiene únicamente 2 elementos (índices 0 y 1).

Al intentar acceder al índice 4, se sale de los límites permitidos para esa fila específica, lo que provoca inmediatamente que Java lance una excepción en tiempo de ejecución: ArrayIndexOutOfBoundsException.
5.- B. Prueba de escritorio
6.- A. **Ordenamiento (`Arrays.sort(fruts)`)**: El método `Arrays.sort()` ordena los elementos de tipo `String` alfabéticamente (orden lexicográfico). Tras la ejecución, el arreglo queda organizado de la siguiente manera:

- Índice `0`: `"Apple"`
- Índice `1`: `"Banana"`
- Índice `2`: `"Orange"`

Búsqueda binaria (`Arrays.binarySearch(fruts, "Banana")`): El método `binarySearch` busca el elemento `"Banana"` dentro del arreglo **ya ordenado**. Como `"Banana"` se encuentra exactamente en la posición (índice) **`1`**, el método devuelve `1`.
7.- D. **El método `Arrays.sort(array, fromIndex, toIndex)`**:

- Este método en Java permite ordenar únicamente un **rango específico** de un arreglo.
- El parámetro `fromIndex` es **inclusivo** y el parámetro `toIndex` es **exclusivo**.

8.- E. **El método System.arraycopy**:
Su sintaxis es System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length):

- src: Arreglo de origen (array).
- srcPos: Índice de origen desde donde se empezarán a leer los elementos (2).
- dest: Arreglo de destino (array, es el mismo arreglo).

- destPos: Índice de destino donde se empezarán a escribir los elementos (1).
- length: Cantidad de elementos que se van a copiar (2).

Ejecución de la copia:
- El arreglo original es {1, 2, 3, 4, 5} (índices 0 al 4).
- A partir del índice 2, los elementos a copiar son 3 (índice 2) y 4 (índice 3). Como la longitud es 2, se copian estos dos valores.
- Estos valores se pegan en el mismo arreglo comenzando en el índice 1:
    - En el índice 1 se escribe 3.
    - En el índice 2 se escribe 4.
- El resto del arreglo permanece sin cambios. El nuevo estado del arreglo es **{1, 3, 4, 4, 5}**.

Evaluación de las impresiones:
- System.out.print(array[1]); → El valor en el índice 1 es **3**.
- System.out.print(array[4]); → El valor en el índice 4 es **5**.
- Al imprimirlos juntos de forma consecutiva, el resultado es **35**.

9.- **a. int[][] array2D = new int[2][2]; array2D[0][0] = 1; ... (Válida)**:
Se instancia correctamente una matriz bidimensional de 2×2 utilizando el operador new, y posteriormente se asignan valores de tipo entero (int) de manera individual a cada una de sus posiciones válidas.
b. int[] array = {0, 1}; int[][][] array3D = new int[2][2][2]; array3D[0][0] = array; ... (Válida):
En un arreglo tridimensional (int[][][]), acceder a dos niveles (como array3D[0][0]) devuelve una referencia a un arreglo unidimensional (int[]). Como array es precisamente un arreglo de tipo int[], la asignación es completamente válida y compila sin problemas.